TRIBHUVAN UNIVERSITY

Faculty of Humanities and Social Sciences

Pashupati Multiple Campus

A Project Report on

ANKA: STUDENT RESULT ANALYSIS AND RANKING SYSTEM

Submitted to

Department of Computer Application

Pashupati Multiple Campus

In partial fulfillment of the requirements for the degree of

Bachelor of Computer Applications (BCA)

Project III (CACS452), Eighth Semester

Submitted by

Niranjan Katwal

TU Symbol No.: 27802063

TU Registration No.: 6-2-278-18-2022

Project Supervisor

Name and designation of the supervisor

Month and year of submission

SUPERVISOR'S RECOMMENDATION

I hereby recommend that this project report, prepared under my supervision by **Niranjan Katwal** (TU Symbol No. 27802063, Registration No. 6-2-278-18-2022) and entitled "**Anka: Student Result Analysis and Ranking System**", be accepted as partial fulfillment of the requirements for the degree of Bachelor of Computer Applications. In my opinion, this report is satisfactory for submission to the evaluation committee.

Signature

Name of the supervisor

Designation

Pashupati Multiple Campus

Date: dd/mm/yyyy

CERTIFICATE OF APPROVAL

This is to certify that the project report entitled "**Anka: Student Result Analysis and Ranking System**" submitted by **Niranjan Katwal** in partial fulfillment of the requirements for the degree of Bachelor of Computer Applications has been examined and approved by the evaluation committee.

Signature Supervisor: Name

Signature Internal Examiner: Name

Signature External Examiner: Name

Signature Head of Department / Program Coordinator: Name

Signature Campus Chief: Name

Date: dd/mm/yyyy

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my project supervisor, name of the supervisor, for the guidance, patient review and constructive suggestions that shaped this project. I am thankful to the Head of Department and all the teachers of the Department of Computer Application at Pashupati Multiple Campus for the knowledge they shared during the four years of the BCA program.

I thank the Tribhuvan University for the project framework that gave this work a clear direction, and my classmates and friends who tested early versions and gave honest feedback. Finally, I am grateful to my family for their constant support and encouragement.

Any remaining mistakes are my own, and I welcome suggestions for improvement.

Niranjan Katwal

ABSTRACT

Educational institutions in Nepal still prepare results from paper registers and spreadsheets. Marks are copied several times, grade points are calculated by hand, back paper results are hard to track, printed marksheets cannot be verified, and weak students are noticed only after the final examination. This project presents **Anka**, a web based Student Result Analysis and Ranking System that removes these problems for institutions of any level.

Anka stores programs, subjects, marks and attendance in a MySQL database and exposes them through a Node.js and Express REST API and a React client. A configurable grading engine supports TU semester GPA, NEB grade points and percentage divisions with credit-weighted SGPA and CGPA. A back paper (re-sit) is stored as a separate sitting so that a cleared failure updates the semester result while the full history is kept. Students are ranked with a merge sort that handles ties, and the system provides analytics based on a binary heap, descriptive statistics, linear regression, Pearson correlation and k-means clustering, all implemented from scratch. An explainable early-warning score identifies students at risk. Marksheets carry a QR code and a ten-character code that anyone can verify. Result, approval, reset, notice and risk emails are sent through a queue with retries. A daily class module provides a timetable with clash detection, a class diary and assignment tracking. The product is unbranded by default, so any school, college or university can use it.

The system was tested with 18 unit tests, 58 API flow checks, 43 smoke checks, 32 classroom checks and a browser end-to-end suite that drives the real pages. The results show that the objectives were met. The grade table and back paper rules follow the commonly published TU scheme and are editable, because they could not be verified against the official regulation.

Keywords: result processing, GPA, back paper, ranking, merge sort, k-means, early warning, REST API, React, MySQL, Nepal

TABLE OF CONTENTS

SUPERVISOR'S RECOMMENDATION i
CERTIFICATE OF APPROVAL ii
ACKNOWLEDGEMENT iii
ABSTRACT iv
TABLE OF CONTENTS v
LIST OF ABBREVIATIONS vii
LIST OF FIGURES viii
LIST OF TABLES ix
CHAPTER 1: INTRODUCTION 1
1.1 Background 1
1.2 Problem Statement 1
1.3 Objectives 2
1.4 Scope and Limitations 3
1.5 Development Methodology 4
1.6 Report Organization 5
CHAPTER 2: BACKGROUND STUDY AND LITERATURE REVIEW 6
2.1 Background Study 6
2.2 Review of Existing Systems 7
2.3 Literature Review 7
2.4 Technology Background 8
CHAPTER 3: SYSTEM ANALYSIS 9
3.1 Requirement Identification 9
3.2 Feasibility Study 11
3.3 Process Modelling 12
3.4 Data Modelling 13
3.5 Object Modelling 14
CHAPTER 4: SYSTEM DESIGN 19
4.1 Architecture Design 19
4.2 Database Design 20
4.3 API Design 21
4.4 Interface Design 22
4.5 Algorithm Design 23
4.6 Security Design 25
4.7 Email Design 26
CHAPTER 5: IMPLEMENTATION AND TESTING 27

5.1 Implementation 27
5.2 Testing 29
5.3 Result Analysis 31
CHAPTER 6: CONCLUSION AND FUTURE RECOMMENDATIONS 33
6.1 Conclusion 33
6.2 Future Recommendations 33
REFERENCES 34
APPENDIX A: SCREENSHOTS 36
APPENDIX B: RUNNING THE SYSTEM 41

LIST OF ABBREVIATIONS

API	Application Programming Interface
BCA	Bachelor of Computer Applications
CGPA	Cumulative Grade Point Average
CORS	Cross-Origin Resource Sharing
CRUD	Create, Read, Update, Delete
CSV	Comma-Separated Values
DFD	Data Flow Diagram
ER	Entity Relationship
GPA	Grade Point Average
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
JSON	JavaScript Object Notation
JWT	JSON Web Token
NEB	Nepal Examination Board
RBAC	Role Based Access Control
REST	Representational State Transfer
SGPA	Semester Grade Point Average
SMTP	Simple Mail Transfer Protocol
SQL	Structured Query Language
TU	Tribhuvan University
UI	User Interface
UML	Unified Modeling Language
QR	Quick Response (code)

LIST OF FIGURES

Figure 3.1: Project schedule (Gantt chart) 12
Figure 3.2: Context diagram (DFD level 0) 12
Figure 3.3: DFD level 1 13
Figure 3.4: Entity relationship diagram 14
Figure 3.5: Use case diagram 15
Figure 3.6: Class diagram of the main classes 16
Figure 3.7: Activity diagram: calculate and publish results 17
Figure 3.8: Sequence diagram: publish results and send email 17
Figure 4.1: System architecture 19
Figure 4.2: Life cycle of a failed subject and its back paper 21
Figure A.1: Landing page 36
Figure A.2: Personal GPA checker with back papers 36
Figure A.3: Marksheet verification result 37
Figure A.4: Administrator dashboard 37
Figure A.5: Programs with grading schemes 37
Figure A.6: Results workflow 38
Figure A.7: Mail center 38
Figure A.8: Marks entry grid 38
Figure A.9: Back paper entry 39
Figure A.10: Classes: weekly timetable 39
Figure A.11: Marksheet with QR code 39
Figure A.12: Student insights and risk 40
Figure A.13: Result email as rendered in a browser 40

LIST OF TABLES

Table 1.1: Development phases and activities 4
Table 2.1: Comparison of existing approaches with Anka 7
Table 2.2: Technologies used and their role 8
Table 3.1: Functional requirements 9
Table 3.2: Non-functional requirements 10
Table 3.3: Estimated cost of development and a first deployment 11
Table 3.4: Use case description: Calculate and publish results 15
Table 3.5: Use case description: Enter back paper marks 15
Table 3.6: States of a result row 18
Table 4.1: Core tables and key columns 20
Table 4.2: Supporting tables 20
Table 4.3: REST endpoints by router 21
Table 4.4: Pages available to each role 22
Table 4.5: TU grade scale used by default (editable per program) 23
Table 4.6: Risk score rules (maximum 100) 25
Table 4.7: Security measures 25
Table 5.1: Tools used 27
Table 5.2: Test suites 29
Table 5.3: Selected functional test cases 30
Table 5.4: Notable defects found during testing 31

CHAPTER 1: INTRODUCTION

1.1 Background

Every term, educational institutions in Nepal collect marks from many teachers, convert them into grades, work out each student's standing, and announce results. In most campuses and schools this chain is still handled with paper mark ledgers, separate spreadsheets for each subject, and a notice board. A teacher writes marks in a register, the examination office copies them into a spreadsheet, a clerk calculates grade points by hand or with fragile formulas, and the final list is printed. Each copy step is a chance for a typing error, and each error becomes visible only after students complain.

Tribhuvan University (TU) uses a semester system for its four-year Bachelor of Computer Applications (BCA) program. Every semester has a set of subjects with credit hours, and each subject is assessed through internal marks and an external (board) examination. A student who fails a subject does not repeat the whole semester. Instead the student sits a *back paper* (a re-sit) in a later examination, and when it is passed the earlier failure is cleared. The Nepal Examination Board (NEB) for grades 11 and 12 and school boards for lower grades use their own grade-point tables and pass rules. A single institution can therefore run several programs with different rules at the same time.

Anka (Nepali for "number" or "mark") is a web based Student Result Analysis and Ranking System built to remove this manual work. It stores syllabus, marks and attendance in one MySQL database, calculates semester GPA and cumulative GPA with a grading engine that can be configured for each program, ranks students with an algorithm implemented from scratch, warns about students at academic risk, sends result emails, and verifies printed marksheets with a QR code. The same platform also records the daily life of a class: the weekly timetable, what was taught in each class, and assignments with submission status.

1.2 Problem Statement

The following problems were identified in the way results and classes are managed in many institutions:

- **Slow and error-prone results.** Marks are copied between registers and spreadsheets by hand. A wrong cell or a wrong formula changes a student's GPA, and the mistake is usually found late.

- **Back papers are hard to track.** A failed subject may be cleared one or two years later. Paper records make it difficult to know who still owes which subject, and a student who passes a back paper cannot easily see the effect on the semester GPA and the cumulative GPA.
- **Different programs need different rules.** A BCA program uses a 4.0 credit-weighted GPA, a higher secondary program uses grade points with a lower pass mark, and an annual bachelor program uses divisions. Most tools support exactly one of these.
- **Forged or disputed marksheets.** A printed marksheet carries no easy way for an employer or another institution to check that it is genuine.
- **Late warning for weak students.** Attendance shortage, falling GPA and failed subjects are noticed only after the final examination, when it is too late to help the student.
- **Poor communication.** Students and guardians learn about results from a notice board, and teachers learn about their new responsibilities by word of mouth.
- **Daily class management is scattered.** Timetables, class diaries and assignment lists live in notebooks and messaging groups, so there is no single record of what was taught and what is due.
- **Software is tied to one institution.** Many local solutions hard-code a name, a logo or one grading rule, so another institution cannot adopt them without changing the source code.

The project therefore asks one practical question: can one low-cost web application let any educational institution record marks and classes once, calculate results correctly under its own rules, and give students, teachers and administrators timely and trustworthy information?

1.3 Objectives

The main objective of the project is to design and implement a web based system that automates result processing, ranking, analysis and daily class management for educational institutions of any level. The specific objectives are:

1. To implement a configurable grading engine with TU semester GPA, NEB grade points and percentage divisions, including credit-weighted SGPA and CGPA.

2. To model back papers as separate sittings so that a cleared failure updates the semester result and the transcript while the history of every attempt is kept.
3. To rank students with a merge sort based algorithm that handles ties fairly, and to provide analytical insights using statistics, regression, correlation and k-means clustering implemented from scratch.
4. To warn about at-risk students with an explainable score built from attendance, failed subjects, GPA trend and internal marks.
5. To provide secure role based access for administrators, teachers, students and the public, with verification of marksheets through a QR code.
6. To send result, approval, password reset, notice and risk emails through a reliable queue with retries and a complete log.
7. To support daily class automation through a timetable, class diary and assignment tracking.
8. To keep the product neutral: no institution name, logo or affiliation is built in, while every institution can add its own details.

1.4 Scope and Limitations

1.4.1 Scope

- Four kinds of users: the public (result search, verification, personal GPA checker), students, teachers and administrators.
- Programs of any level (school grades, higher secondary, bachelor, master) with their own term names, term counts, pass percentages and grading rules.
- Marks entry by typing or CSV import, attendance with an 80% eligibility rule, back paper entry, result calculation, publication, promotion to the next term, and a transcript with backlogs.
- Result analytics, merit lists, early-warning risk scores, cohort insights and an algorithm explorer page.
- A mail center with a queue, retries, previews and a log; notices; an audit log; optional institution branding.
- Daily class tools: weekly timetable with clash detection, class diary, assignments and submission tracking.

1.4.2 Limitations

- The system is a web application. A native mobile application is not part of this work, although every page works on a phone screen.
- The grade table and back-paper rules follow the commonly published TU scheme, but they could not be verified against the official regulation document. They are stored as editable data, so a campus can match its own rules.
- One installation serves one institution. Several institutions on one server (multi-tenancy) is future work.
- Assignment files are not uploaded; the system records the due date, status, marks and feedback only.
- Email delivery depends on a Gmail account with an app password configured by the administrator.
- Online payment, fee management and biometric attendance are outside the scope.

1.5 Development Methodology

The project followed an incremental and iterative process. A first working prototype was reviewed, and then each cycle added one slice of functionality, tested it, and fixed what the tests exposed. The work moved through requirement analysis, design, implementation and testing in repeated short cycles, which suits a single developer who must keep the whole system consistent.

Table 1.1: Development phases and activities

Phase	Activities	Output
Requirement identification	Study of the result process, TU syllabus for Project III, grading rules and existing systems	Requirement list, use cases
Analysis	Feasibility study, DFD, ER diagram, class diagram	Analysis models
Design	Architecture, database schema, API design, interface design with a design system	Schema, route list, UI components
Implementation	Node.js API, React client, grading and ranking algorithms, mail queue	Working system in Git repository
Testing	Unit tests, API flow tests, smoke tests, browser end-to-end tests	Test suites and results
Documentation	Proposal, report, presentation	This document

Version control was done with Git and every logical step was committed to a public GitHub repository. The database is MySQL 8.4 and a portable copy is kept inside the project folder, so the system can be started on any Windows computer without installing a database server.

1.6 Report Organization

This report has six chapters. Chapter 1 introduces the problem, objectives and scope. Chapter 2 reviews existing systems and the theory behind the algorithms used. Chapter 3 analyzes the system through requirements, feasibility, and process, data and object models. Chapter 4 describes the design: architecture, database, interface and algorithms. Chapter 5 explains the implementation and testing and reports the test results. Chapter 6 concludes the work and lists future recommendations. References and appendices with screenshots follow.

CHAPTER 2: BACKGROUND STUDY AND LITERATURE REVIEW

2.1 Background Study

2.1.1 The TU Semester System and GPA

Under the TU semester system a program is divided into terms, each with subjects that carry credit hours. Marks are given out of a full mark that is split into an internal part (assessment during the term) and an external part (the board examination). The percentage obtained in a subject is converted into a letter grade and a grade point on a 4.0 scale. The semester grade point average (SGPA) is the credit-weighted mean of the grade points of the subjects of a semester, and the cumulative grade point average (CGPA) is the credit-weighted mean over all semesters:

```
SGPA = sum(grade_point_i x credit_i) / sum(credit_i)
CGPA = sum(SGPA_s x credits_s) / sum(credits_s)
```

A subject is passed only when the student obtains at least the pass percentage in the subject as a whole and in the external component. In Anka these values are properties of a program, so a 40% rule for BCA and a 35% rule for a higher secondary program can live side by side.

2.1.2 Back Papers

A student who fails a subject may appear in a later examination for that subject only. The result of the new sitting replaces the failed one when it is a pass. The cohort to which the student belongs, the semester in which the subject was taught, and the earlier attempts must all remain intact. This is why Anka stores each sitting as its own row with an attempt number and chooses the *effective attempt* when a result is calculated: the first passing sitting if there is one, otherwise the most recent sitting.

2.1.3 Education Systems of Other Levels

The NEB uses grade points from 1.6 (grade D) to 4.0 (grade A+) with a pass mark of 35% in each subject, and school levels often use divisions (Distinction, First, Second, Pass) derived from the total percentage. Annual bachelor programs under TU also report divisions. A reusable result system must therefore treat the grading rule as data, not as code. Anka provides three schemes: `tu_gpa`, `neb_gpa` and `division`.

2.2 Review of Existing Systems

Several kinds of systems are used for results and student records. They solve parts of the problem but leave gaps for a small or medium institution.

Table 2.1: Comparison of existing approaches with Anka

Approach	Strengths	Gaps for a campus or school
Paper ledger and spreadsheet	No cost, familiar	Manual GPA, copy errors, no back paper history, no verification, no analytics
University result portal (TU examination results)	Authoritative result for the university examination	Shows the final result only; does not cover internal marks, attendance, classes or institution-level analysis
Board result portals and SMS (NEB)	Wide reach for board results	No teacher workflow, no early warning, no daily class tools
Open-source school software (for example OpenSIS, Fedena)	Feature rich, community support	Grading rules are not Nepal specific, heavy to install and configure, no back paper model
Learning management systems (for example Moodle)	Course content, quizzes, submissions	Not designed for credit-weighted GPA, ranking and result publication
Anka (this project)	Configurable grading, back papers, ranking, risk score, verified marksheets, email, daily classes, unbranded	Single institution per installation, no file uploads

2.3 Literature Review

Educational data mining studies how data collected by educational institutions can be turned into information that helps students and teachers. Romero and Ventura [1] review the field and list prediction of performance, grouping of students and the detection of undesirable behavior among the main tasks. Anka follows the same direction in a deliberately simple and explainable form: a regression forecast of the next GPA, k-means grouping of students, and a risk score whose reasons are shown to the user.

Early warning for students at risk has been reported to depend on a small number of strong indicators such as attendance, previous results and assessment scores. Using weights that are visible to the teacher avoids the "black box" problem: a teacher can see why a student was flagged and discuss it with the student. This is the design choice behind the risk score in Section 4.5.

For ranking, the stable merge sort described by Cormen et al. [2] runs in O(n log n) time in the worst case and keeps equal elements in their original order, which matters when

students with equal GPA must be listed consistently. A binary heap [2] gives the top k students in O(n log k) without sorting everyone. For grouping, the k-means algorithm [3], [4] partitions points into k clusters by repeatedly assigning points to the nearest center and recomputing centers. The Pearson correlation coefficient [5] measures the linear relation between two variables, for example attendance and GPA, and simple linear regression by least squares gives the trend line used for forecasting.

For security, passwords are hashed with bcrypt, an adaptive function designed by Provos and Mazieres [6], and sessions use JSON Web Tokens as defined in RFC 7519 [7]. Emails are sent with the Simple Mail Transfer Protocol described in RFC 5321 [8].

2.4 Technology Background

Table 2.2: Technologies used and their role

Technology	**Role in the system**
React 18 and Vite	Component based client application and fast build tool
Framer Motion, Recharts, three.js, qrcode.react	Interface motion, charts, 3D scenes on public pages, QR code of marksheets
Node.js and Express	REST API, authentication middleware, role checks
MySQL 8.4 with mysql2	Relational storage with transactions and foreign keys
bcryptjs and jsonwebtoken	Password hashing and signed session tokens
Nodemailer with Gmail SMTP	Email delivery through an application password
puppeteer-core (Microsoft Edge)	Browser end-to-end tests

CHAPTER 3: SYSTEM ANALYSIS

System analysis studies the existing way of working, identifies what the new system must do, checks that it can be built with the available time, skills and tools, and models the system from the process, data and object points of view.

3.1 Requirement Identification

3.1.1 Study of the Existing System

The existing process in a typical campus was studied through the project syllabus, the published TU examination rules and the experience of a student who has taken part in the process for four years. It works in five manual steps: (1) subject teachers record internal marks in a register; (2) the board examination marks arrive on paper; (3) the examination office copies all marks into a spreadsheet and calculates grades; (4) a printed result is put on the notice board and marksheets are typed one by one; (5) failed students are noted down for the next back paper examination. The weaknesses are repeated data entry, calculation mistakes, lack of history, no trend analysis, and no verification of marksheets.

3.1.2 Requirement Collection

Requirements were collected by studying the result cycle of a semester and by writing user stories for each role. They were grouped into functional requirements (what the system does) and non-functional requirements (how well it does it).

3.1.3 Functional Requirements

Table 3.1: Functional requirements

ID	Requirement	Actor
FR1	Register, sign in, sign out, change and reset a password by email link	All users
FR2	Approve or reject new teacher accounts	Administrator
FR3	Create and edit programs with grading scheme, pass percentage, term names and term count	Administrator
FR4	Manage subjects with credit, internal and external full marks, and assigned teacher	Administrator
FR5	Manage students with symbol number, program, term, batch and guardian email	Administrator
FR6	Enter marks by typing or CSV import, with validation against the full marks	Teacher
FR7	Enter back paper marks for students who have not cleared a subject	Teacher
FR8	Take attendance with present, late and absent; compute the 80% eligibility	Teacher

ID	Requirement	Actor
FR9	Calculate results for a program, term and exam year, rank the cohort, and publish or unpublish	Administrator
FR10	Promote students to the next term and keep the history of earlier terms	Administrator
FR11	Show marksheet with QR code, transcript with CGPA and backlogs, and attendance	Student
FR12	Search a result by symbol number and verify a marksheet by code without signing in	Public
FR13	Check a personal GPA with back papers using editable grade bands	Public, student
FR14	Show student insights: GPA forecast, percentile, risk score and reasons	Student, teacher
FR15	Show class analytics: statistics, top performers, clusters, correlation	Teacher, administrator
FR16	Send result, approval, reset, notice and risk emails through a queue; log and preview every message	System, administrator
FR17	Publish notices for chosen audiences	Administrator
FR18	Record an audit log of important actions	System
FR19	Maintain a weekly timetable and reject clashes	Teacher, administrator
FR20	Record what was taught in each class (class diary)	Teacher
FR21	Create assignments and track submitted, late and missing work with marks	Teacher
FR22	Show today's classes and due assignments to students	Student
FR23	Allow optional institution branding on marksheets and emails	Administrator

3.1.4 Non-functional Requirements

Table 3.2: Non-functional requirements

Quality	Requirement
Security	Passwords hashed with bcrypt; JWT sessions; role checks on every route; rate limiting on public lookups; no sensitive fields in public data
Accuracy	GPA and ranking must equal hand calculation; marks are validated before saving; results are calculated inside database transactions
Usability	Clear navigation, command palette, keyboard friendly forms, light and dark themes, responsive down to phone width
Performance	Common pages respond within one second for a cohort of a few hundred students; ranking in O(n log n)
Reliability	Email queue with up to three attempts; published results stay published after back paper recalculation
Portability	Runs on Windows, Linux or macOS with Node.js and MySQL; portable MySQL copy provided

Quality	Requirement
Maintainability	Grading scales and rules are data; one Express router per concern; automated tests
Neutrality	No built-in institution name or branding; every label is editable

3.2 Feasibility Study

3.2.1 Technical Feasibility

Every technology used is open source, well documented and runs on an ordinary laptop: Node.js, Express, React, Vite and MySQL. The developer has prior experience with JavaScript and relational databases. A portable MySQL copy removes installation difficulty. The project is therefore technically feasible.

3.2.2 Operational Feasibility

The system replaces spreadsheets and paper but keeps the same roles and the same sequence of work. Teachers enter marks in a grid that behaves like a spreadsheet, with CSV import for existing sheets. Students use the same symbol number they already know. The interface needs no training beyond a short walk-through. Operation is feasible.

3.2.3 Economic Feasibility

Table 3.3: Estimated cost of development and a first deployment

Item	Cost
Software licences (Node.js, React, MySQL, VS Code, Git)	NPR 0 (open source)
Developer effort (one student, 17 weeks, part time)	Academic work
Gmail account with app password for email	NPR 0
Hosting for a small campus (virtual server or on-premise PC)	About NPR 6,000 to 15,000 per year, or existing computer
Domain name (optional)	About NPR 1,500 per year

The system costs almost nothing to build, and the running cost is small compared with the staff time saved every examination cycle. The project is economically feasible.

3.2.4 Schedule Feasibility

The work was planned over seventeen weeks. Figure 3.1 shows the schedule. Because the work was divided into small increments, a delay in one area (for example the frontend design) did not block the others.

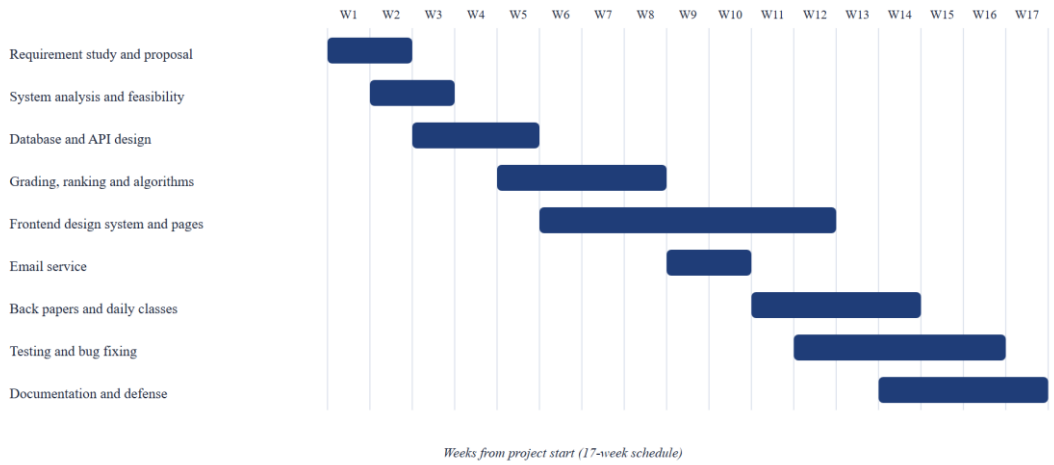


Figure 3.1: Project schedule (Gantt chart)

3.3 Process Modelling

Data flow diagrams (DFD) show how data moves between the external entities, the processes and the data stores.

3.3.1 Context Diagram (DFD Level 0)

Figure 3.2 shows the whole system as a single process. The administrator supplies the academic setup and publication decisions, teachers supply marks, attendance and class records, students and the public receive results and insights, and guardians receive result emails.

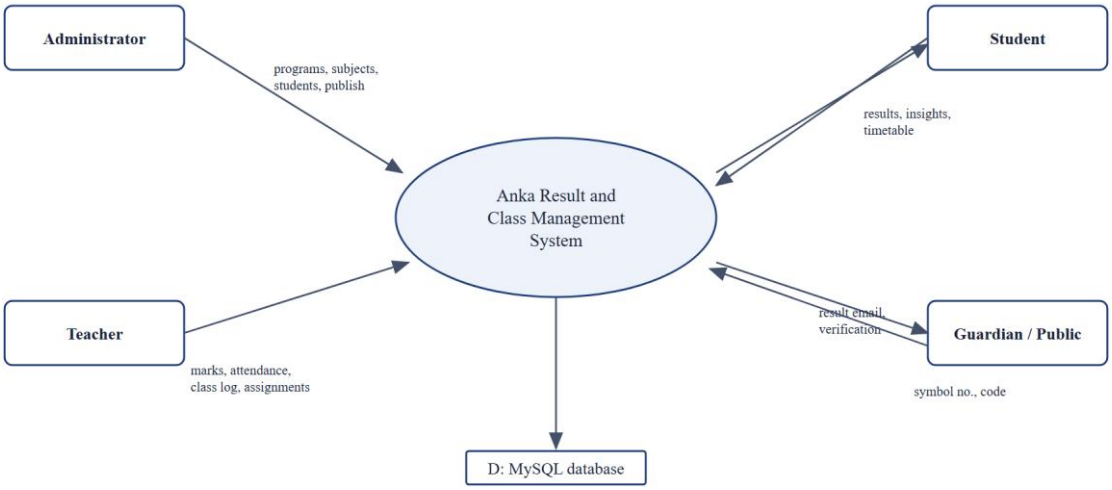


Figure 3.2: Context diagram (DFD level 0)

3.3.2 DFD Level 1

Figure 3.3 breaks the system into six processes: managing the academic setup, recording marks and back papers, calculating results, publishing and notifying, analyzing and ranking, and daily class automation.

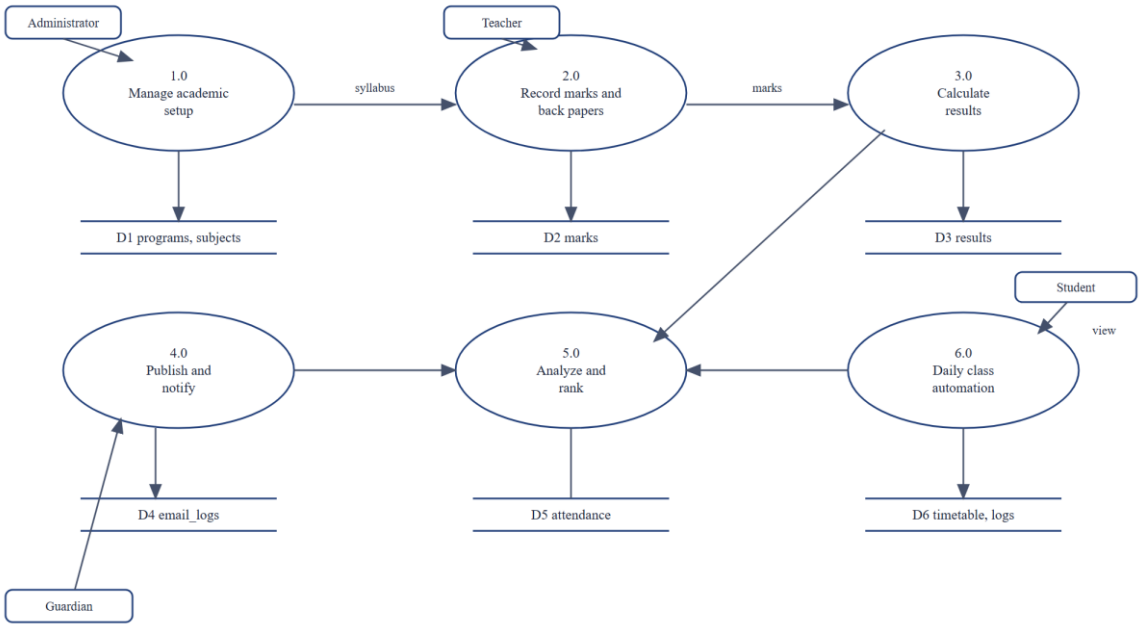


Figure 3.3: DFD level 1

3.4 Data Modelling

The database has sixteen tables. The core entities are `programs`, `users`, `teachers`, `students`, `subjects`, `marks`, `results` and `attendance`. The daily class module adds `timetable`, `class_logs`, `assignments` and `assignment_status`. The supporting tables are `notices`, `email_logs`, `audit_logs` and `settings`. Figure 3.4 shows the entity relationship diagram.

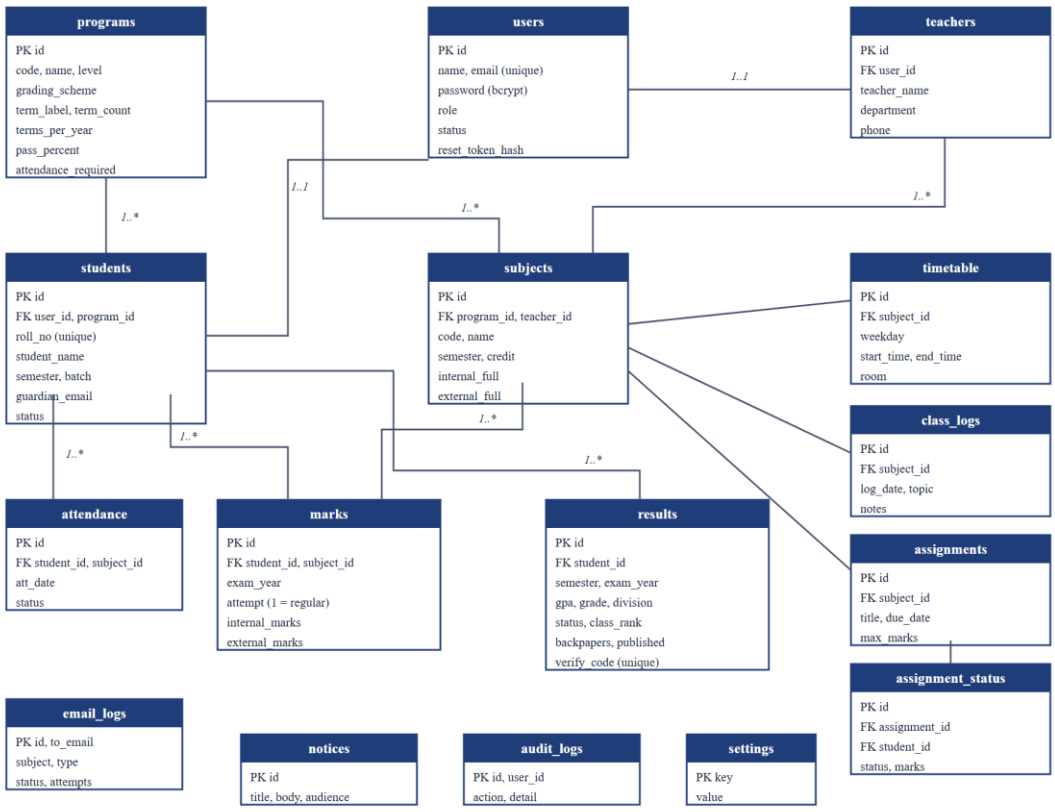


Figure 3.4: Entity relationship diagram

Three design decisions deserve attention. First, `marks` has an `attempt` column, so a back paper is another row for the same student and subject. Second, `results` stores a `backpapers` count and a unique `verify_code` that is printed on the marksheet. Third, the program row carries the term label, term offset and terms per year, so that "Semester 7", "Year 3" and "Grade 11" are all produced from data.

3.5 Object Modelling

3.5.1 Use Case Diagram

Figure 3.5 shows the use cases of the four actors. The public visitor can look up and verify results and use the GPA checker. Students see their own marksheets, attendance and insights. Teachers manage marks, back papers, attendance and classes of their own subjects only. The administrator manages the institution and the result cycle.

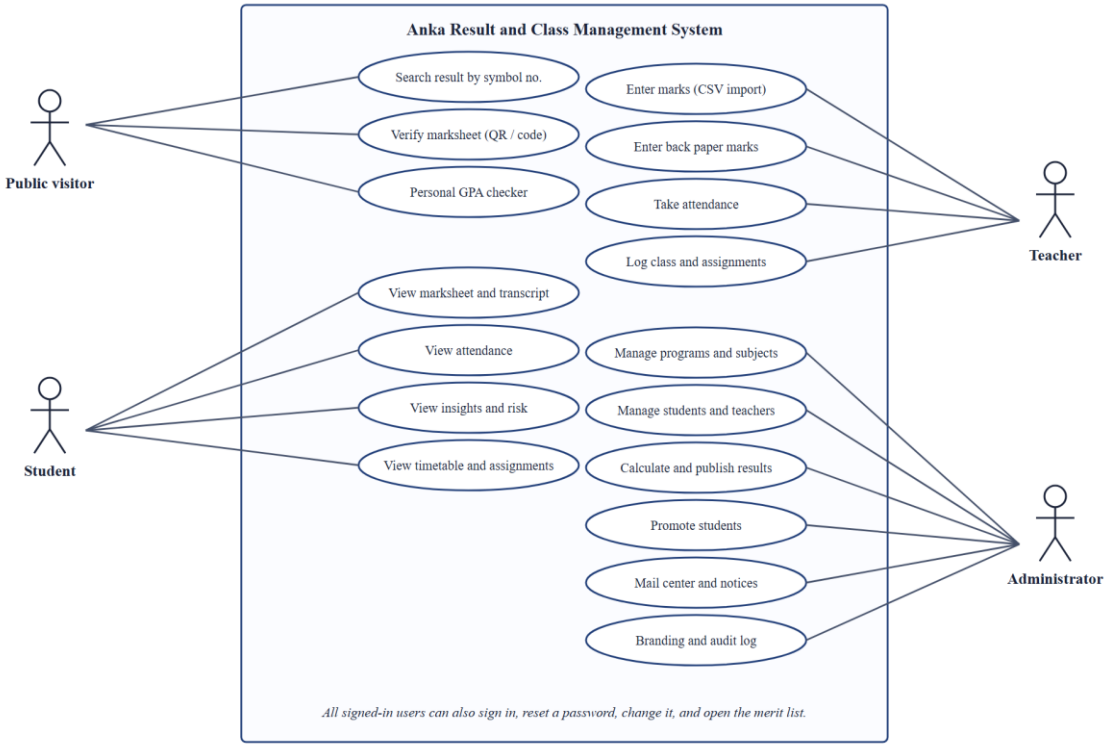


Figure 3.5: Use case diagram

Table 3.4: Use case description: Calculate and publish results

Item	Description
Actor	Administrator
Precondition	Marks for the term are entered; the administrator is signed in
Main flow	1. Choose program, term and exam year. 2. System calculates subject grades, SGPA, status and class rank. 3. Administrator reviews the list and publishes. 4. System queues result emails to students and guardians.
Alternative flow	If a student has a back paper pass, the effective attempt is used and the result is recalculated without changing the published state.
Postcondition	Results are saved and visible to students; emails are logged

Table 3.5: Use case description: Enter back paper marks

Item	Description
Actor	Teacher
Precondition	The subject is assigned to the teacher; at least one student has not cleared it
Main flow	1. Open Marks entry and choose Back papers. 2. System lists students who still owe the subject with every earlier sitting. 3. Teacher enters marks for the chosen year and saves. 4. System stores a new attempt and recalculates the original semester result.
Alternative flow	Marks above the full mark are rejected with a message.
Postcondition	A passing sitting clears the backlog; GPA and CGPA are updated

3.5.2 Class Diagram

Figure 3.6 shows the main classes. The grading engine and result service work on `Marks`, `Subject` and `Program` and produce `Result` objects; the mail service is called by the result service when a result is published.

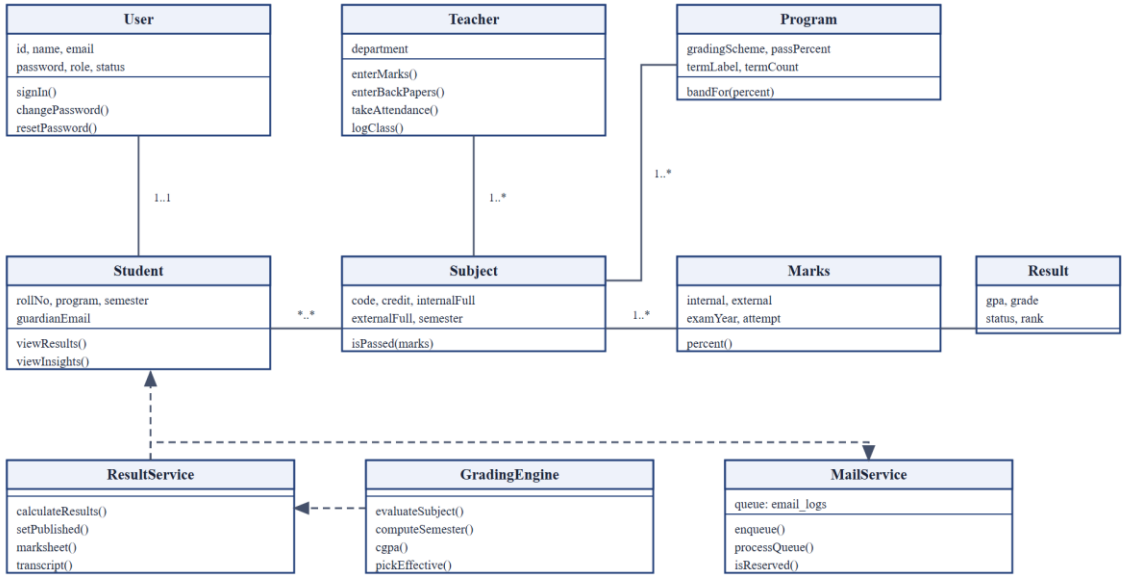


Figure 3.6: Class diagram of the main classes

3.5.3 Activity Diagram

Figure 3.7 shows the activity of calculating and publishing results, including the choice of the effective attempt.

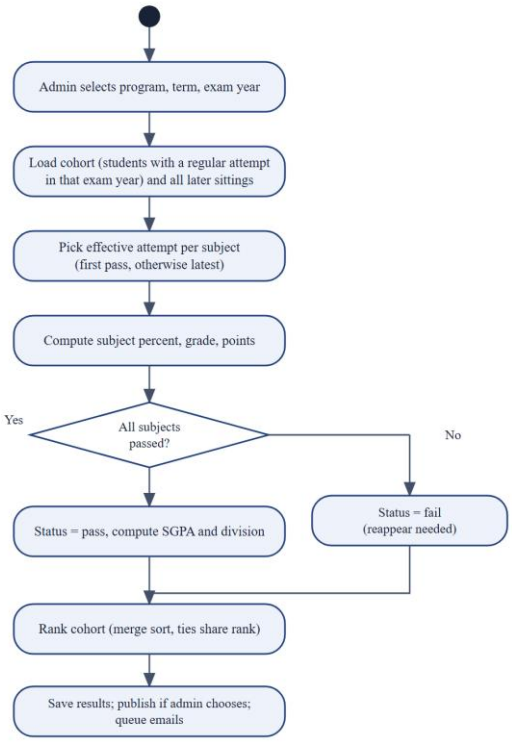


Figure 3.7: Activity diagram: calculate and publish results

3.5.4 Sequence Diagram

Figure 3.8 shows what happens when the administrator publishes results. The API publishes inside a transaction and only queues the emails; a separate worker sends them, so a slow mail server never blocks the administrator.

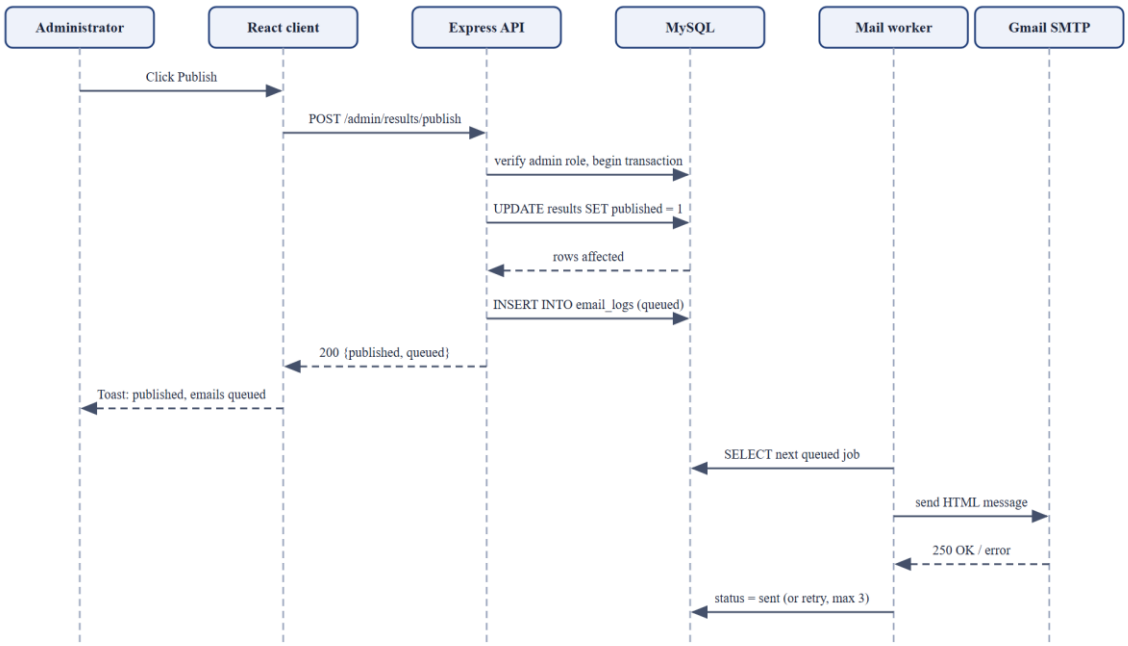


Figure 3.8: Sequence diagram: publish results and send email

3.5.5 State of a Result

Table 3.6: States of a result row

State	Meaning	Moves to
Draft	Calculated but hidden from students	Published, recalculated draft
Published	Visible to students, verifiable by code	Draft (unpublish), recalculated and still published after a back paper
Fail	At least one subject not passed (reappear needed)	Pass after back papers are cleared
Pass	All subjects passed on their effective attempt	None

CHAPTER 4: SYSTEM DESIGN

The design translates the analysis into an architecture, a database schema, an interface, and the algorithms that produce results. This chapter follows the same order.

4.1 Architecture Design

Anka uses a three-tier architecture. The presentation tier is a single page React application. The application tier is a stateless Node.js and Express REST API that checks the JSON Web Token on every request and applies the role of the user. The data tier is a MySQL database. The mail service is a worker inside the API process that reads queued messages from the database and sends them through Gmail SMTP.

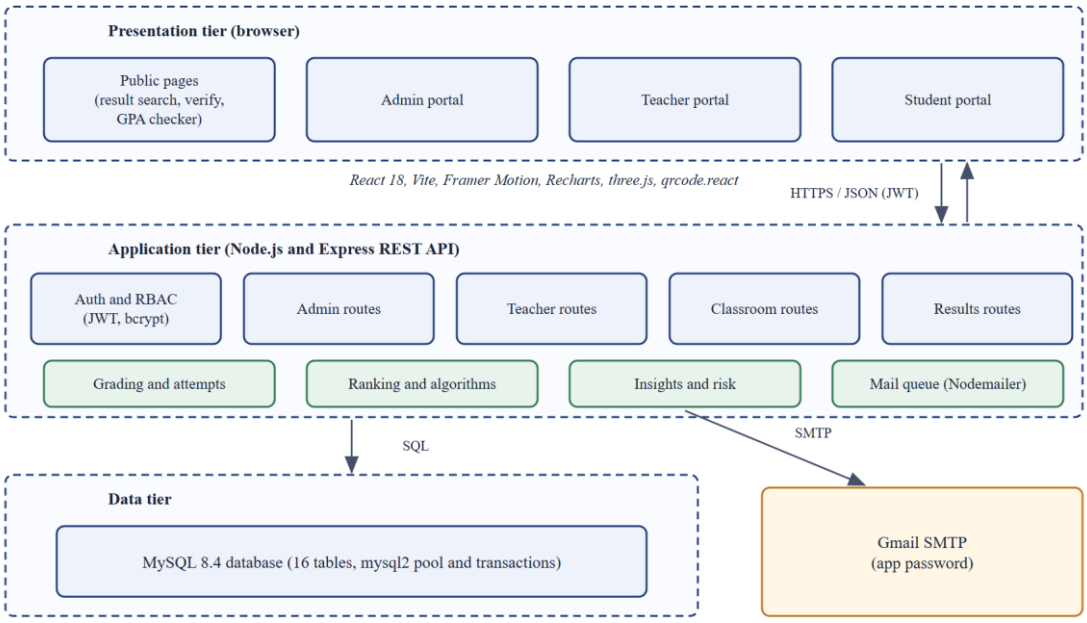


Figure 4.1: System architecture

Keeping the API stateless means it can be restarted at any time without losing sessions, and the same API can later serve a mobile client. Business rules live in service modules (`resultService`, `marksService`, `grading`, `attempts`, `ranking`, `insights`, `mailer`) and not in the route handlers, so each rule can be unit tested without a web server.

4.2 Database Design

4.2.1 Main Tables

Table 4.1: Core tables and key columns

Table	Purpose	Key columns
programs	Course of study and its rules	code, name, level, grading_scheme, term_label, term_count, term_offset, terms_per_year, pass_percent, attendance_required
users	Login accounts	email (unique), password (bcrypt hash), role, status, reset_token_hash, reset_expires
teachers	Teacher profile	user_id, teacher_name, department, phone
students	Student profile	user_id, program_id, roll_no (unique), semester, batch, guardian_email, status
subjects	Syllabus entry	program_id, teacher_id, code, name, semester, credit, internal_full, external_full
marks	One sitting of one subject	student_id, subject_id, exam_year, attempt, internal_marks, external_marks
results	Calculated term result	student_id, semester, exam_year, gpa, grade, division, status, class_rank, backpapers, published, verify_code
attendance	Daily attendance	student_id, subject_id, att_date, status

Table 4.2: Supporting tables

Table	Purpose
timetable	Weekly class slots: subject, weekday, start and end time, room
class_logs	Class diary: one entry per subject per date with topic and notes
assignments, assignment_status	Assignments and per-student status (submitted, late, missing), marks and feedback
notices	Announcements with an audience (all, student, teacher)
email_logs	Every outgoing message with status (queued, sent, failed, skipped), attempts and error
audit_logs	Who did what and when
settings	Optional institution branding values

4.2.2 Integrity Rules

- Foreign keys with `ON DELETE CASCADE` remove dependent rows (for example the marks of a deleted student); authorship columns use `SET NULL` so that deleting a user does not delete history.
- Unique keys prevent duplicates: one account per email, one student per symbol number, one marks row per student, subject, exam year and attempt, one diary entry per subject and date, one status per assignment and student.

- A check constraint limits the timetable weekday to 0 to 6.
- Result calculation and publication run inside transactions, so a failure leaves no half-written result.

4.2.3 Back Paper Data Model

A back paper is a second row in `marks` with `attempt` 2 or higher and a later `exam_year`. The cohort of a result is the set of students having a regular attempt (attempt 1) in the result's exam year; later sittings of those students are included when the effective attempt is chosen. Figure 4.2 summarizes the life of a failed subject.

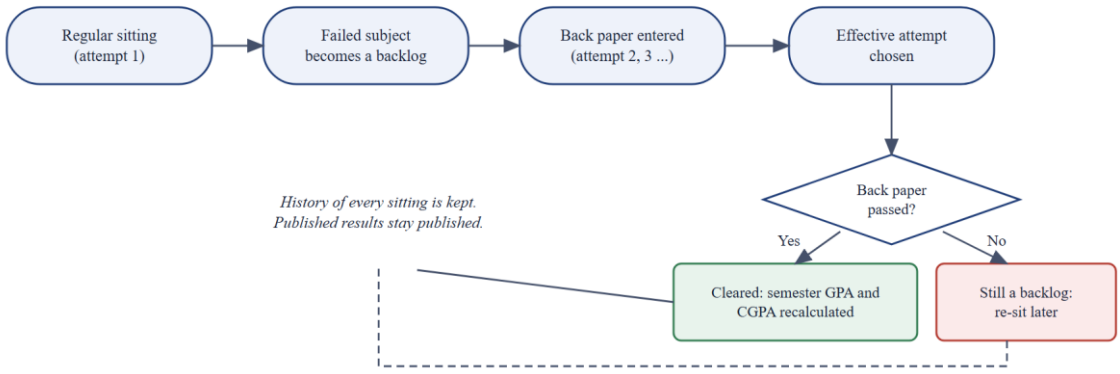


Figure 4.2: Life cycle of a failed subject and its back paper

4.3 API Design

The API is organized as five routers mounted under `/api`. There are 82 route handlers in total. Table 4.3 lists the main endpoints of each router. All endpoints except the public ones require a valid token, and administrator, teacher and student routes check the role.

Table 4.3: REST endpoints by router

Router	Count	Main endpoints
/api/auth	8	POST /register, POST /login, GET /me, PUT /password, POST /forgot-password, POST /reset-password, GET /programs, GET /institution
/api/admin	36	programs, subjects, students and teachers CRUD; POST /results/calculate, /results/publish, /results/promote; GET /results/export.csv; insights and alerts; notices; audit; settings; mail (list, preview, test, retry)
/api/teacher	11	GET and PUT /subjects/:id/marks; POST /marks/import; GET and PUT /backpapers; GET and PUT /attendance; GET /analytics; GET /insights
/api/results	16	public search, marksheet, verify, stats; me/overview, marksheet, transcript, attendance, insights; ranking; notices
/api/classroom	11	GET and POST /timetable; GET /today; GET /diary; PUT /logs; assignments CRUD and status tracking

Responses use one shape: `{ success: true, ... }` on success and `{ success: false, message }` with a suitable status code (400 invalid input, 401 not signed in, 403 not allowed, 404 not found, 409 conflict, 429 too many requests) on failure. The message is written for the user and never exposes internal details.

4.4 Interface Design

4.4.1 Design Principles

- **Neutral and professional.** A restrained navy palette with brass as the accent, no decorative clutter, and typography with a serif for headings on marksheets and a sans-serif for the interface.
- **One design system.** Shared tokens for color, spacing, radius and shadow; shared components for cards, tables, chips, dropdowns, segmented controls, drawers, modals and toasts. Light and dark themes use the same tokens.
- **Precision for data work.** Marks entry behaves like a spreadsheet: Enter moves down a column, Ctrl+S saves, invalid cells are marked, unsaved changes are shown.
- **Orientation.** A collapsible sidebar grouped by task, breadcrumbs, a command palette opened with Ctrl+K, and a notification menu.
- **Responsive and accessible.** Layouts work from desktop to 390 pixel phones; controls have labels; focus is visible; motion is short.

4.4.2 Navigation by Role

Table 4.4: Pages available to each role

Role	Pages
Public	Landing, Result search, Verify marksheet, GPA checker, Algorithms explorer, Sign in, Register, Forgot and reset password
Student	Dashboard, Classes, Results (marksheets and transcript), Attendance, Insights, GPA checker, Merit list, Account
Teacher	My subjects, Classes, Marks entry (regular and back papers), Attendance, Class insights, Merit list, Account
Administrator	Dashboard, Insights, Programs, Subjects, Students, Teachers, Classes, Results, Merit list, Mail center, Notices, Branding, Audit log, Algorithms

4.5 Algorithm Design

All algorithms listed here are implemented from scratch in the `backend/utils` folder without a library, and they are explained interactively on the `/algorithms` page of the running system.

4.5.1 Grade Lookup by Binary Search

A grading scale is an ascending list of bands, each with a minimum percentage, a grade and a grade point. To find the band of a percentage the algorithm searches for the last band whose minimum is not greater than the percentage. Binary search does this in O(log n).

```
bandFor(scale, percent):
  lo = 0; hi = len(scale) - 1
  while lo < hi:
    mid = (lo + hi + 1) div 2
    if scale[mid].min <= percent: lo = mid
    else: hi = mid - 1
  return scale[lo]
```

Table 4.5: TU grade scale used by default (editable per program)

Percentage	Grade	Grade point
90 and above	A	4.0
80 to below 90	A-	3.7
75 to below 80	B+	3.3
70 to below 75	B	3.0
65 to below 70	B-	2.7
60 to below 65	C+	2.3
55 to below 60	C	2.0
50 to below 55	C-	1.7
45 to below 50	D+	1.3
40 to below 45	D	1.0
Below 40, or below pass mark in the external part	F	0.0

4.5.2 Subject, Semester and Cumulative Results

For every subject the engine computes the percentage from internal and external marks, checks the pass rule (overall percentage and external percentage both at least the program pass mark), looks up the grade and grade point, and then computes the semester GPA as the credit-weighted mean. A semester is *pass* only when every subject is passed. The CGPA is the credit-weighted mean of the semester GPAs. For the `division` scheme the GPA is the percentage on a 4.0 scale and the division comes from the total percentage (Distinction 80, First 60, Second 48, Pass 40).

4.5.3 Effective Attempt for Back Papers

```
pickEffective(attempts):
  sort attempts by (exam_year, attempt)
  for each attempt: evaluate subject
  i = index of first passing attempt, else last
  return attempts[i] with history and flag
         cleared_by_backpaper = (i > 0 and passed)
```

The first passing sitting is chosen so that a student who passes a re-sit but later appears again does not lose the pass. If no sitting passes, the latest one is used so that the most recent marks are shown.

4.5.4 Ranking by Merge Sort

Students of a cohort are ordered by GPA, then by total marks, then by symbol number for stability. Merge sort divides the list, sorts both halves and merges them; it is O(n log n) in every case. Equal students receive the same rank (competition ranking: 1, 2, 2, 4), which is the fair choice for equal scores.

4.5.5 Top-k Selection with a Binary Heap

The top performers of a subject are found with a min-heap of size k. Each item is compared with the smallest of the current k; if it is larger it replaces it. The cost is O(n log k), which is cheaper than sorting everyone when k is small.

4.5.6 Statistics, Percentile and Z-score

Descriptive statistics (mean, median, standard deviation, minimum, maximum, pass rate) are computed in a single pass plus a sort for the median. The percentile of a student is found by binary search in the sorted marks, and the z-score is (x - mean) / standard deviation.

4.5.7 Linear Regression and Forecast

The trend of GPA across terms is fitted by least squares: slope = (n*sum(xy) - sum(x)*sum(y)) / (n*sum(x^2) - (sum(x))^2). The next GPA is forecast by extending the line and is clamped between 0 and 4. The slope also feeds the risk score.

4.5.8 Pearson Correlation

The correlation between attendance and GPA (or internal and external marks) is r = cov(x, y) / (sd(x) * sd(y)). A value close to +1 tells the teacher that students who attend more also score more.

4.5.9 K-means Clustering

Students are grouped into three clusters by their GPA and attendance. Both features are scaled to the range 0 to 1 first. The algorithm starts from deterministic centers chosen farthest-first (so the same data always gives the same groups), assigns each student to the nearest center, recomputes the centers as means, and repeats until no assignment changes or fifty iterations have run. The clusters are shown to teachers as support groups.

4.5.10 Explainable Risk Score

Table 4.6: Risk score rules (maximum 100)

Condition	Points	Reason shown to the user
Attendance below 80%	up to 35	Attendance X% is below the 80% exam-eligibility requirement
Failed subjects	12 each, up to 30	N failed subject(s) to clear
GPA slope below -0.1	up to 20	GPA is trending downward over recent terms
Latest GPA below 2.2	10	Latest GPA is low
Internal average below 50%	10	Internal assessment average is weak

A score of 55 or more is *high* risk, 25 to 54 is *medium* and below 25 is *low*. Because every point has a stated reason, a teacher can explain the result to the student.

4.5.11 Tokens and Email Queue

Password reset links use a random token whose SHA-256 hash is stored with an expiry of thirty minutes, so a copy of the database does not reveal a usable link. Outgoing emails are first written to `email_logs` with status queued. A worker takes the oldest queued message, sends it, and marks it sent. On failure it waits (1.5 seconds times the attempt number) and retries until three attempts are used, then marks it failed. Demo addresses on reserved domains are never delivered and are logged as skipped.

4.6 Security Design

Table 4.7: Security measures

Threat	Measure
Stolen database	bcrypt password hashes (cost 10); reset tokens stored only as hashes
Unauthorized access	JWT on every private route; role check per router; teachers can touch only their own subjects (HTTP 403 otherwise)
Brute force and scraping	Per-address rate limit on public lookups (40 per minute by default); generic message for wrong credentials
Injection	Parameterized queries everywhere; input length and range validation

Threat	Measure
Forged marksheets	Unique ten-character code and QR code verified against the database
Accidental data loss	Confirmation dialogs; cascade rules; published results locked against edits
Secret leakage	Credentials only in `backend/.env`, which is excluded from version control

4.7 Email Design

Every email is an HTML message with inline styles (mail clients ignore style sheets), a navy header that shows the institution name only when one is configured, a content card, a key-value table for results, one clear button, and a plain footer. Eight templates exist: welcome, password reset, teacher approved, result, guardian result, risk alert, notice and test. The same palette as the web application keeps the experience consistent.

CHAPTER 5: IMPLEMENTATION AND TESTING

5.1 Implementation

5.1.1 Development Environment

Table 5.1: Tools used

Item	Tool
Operating system	Windows 11
Editor and version control	Visual Studio Code, Git, GitHub
Runtime	Node.js 24
Database	MySQL 8.4 (portable copy in the project folder, port 3308)
Server libraries	express, mysql2, bcryptjs, jsonwebtoken, nodemailer, cors, morgan
Client libraries	react, react-router-dom, vite, framer-motion, recharts, three, qrcode.react, lucide-react
Testing	node:test, fetch based API suites, puppeteer-core with Microsoft Edge
Diagrams and documents	SVG diagrams rendered by Edge; docx and pptxgenjs for documents

5.1.2 Project Structure

```
StuResSystem/
  backend/
    server.js            Express app and router mounting
    routes/              auth, admin, teacher, results, classroom
    utils/               grading, attempts, ranking, algorithms, insights,
                         resultService, marksService, mailer, emailTemplates
    db/                  schema.sql, setup.js (schema and seed data)
    tests/               unit tests, API flow tests, smoke, classroom tests
  frontend/
    src/pages/           public, admin, teacher, student, Classes
    src/components/      Shell, CommandPalette, Marksheet, Scenes, ui
    src/styles.css       design system tokens and components
  e2e/                   browser end-to-end tests
  .mysql/                portable MySQL (not in Git)
```

The backend has about 2,200 lines, the frontend about 4,600 lines of JavaScript, JSX and CSS, and the automated tests about 1,500 lines, so the repository holds roughly 8,500 lines in total.

5.1.3 Grading Engine

The grading engine (`grading.js`) exposes `gradeFor`, `evaluateSubject`, `computeSemester` and `cgpa`. The three scales are plain arrays of bands, so a campus can change a band without changing the algorithm. The excerpt below shows how a subject is evaluated.

```
function evaluateSubject(row, scheme, passPercent) {
  const full = internal_full + external_full;
  const total = internal_marks + external_marks;
  const percent = full ? (total / full) * 100 : 0;
  const extOk = external_full === 0 ||
               (external_marks / external_full) * 100 >= passPercent;
  const passed = percent >= passPercent && extOk;
  const band = passed ? gradeFor(percent, scheme) : SCALES[scheme][0];
  return { total, full, percent, passed, grade: band.grade, point: band.point
};
}
```

5.1.4 Back Paper Processing

`attempts.js` groups the sittings of each student and subject and selects the effective attempt. `resultService.cohortAttempts` loads the regular attempts of the result year and all later sittings, `calculateResults` evaluates them, and `setPublished` changes the published flag. When a teacher saves back paper marks, `marksService.saveBackPapers` inserts the new attempt, and the service recalculates the original semester result for every affected student. A result that was already published stays published, so a student sees the improved GPA immediately.

5.1.5 Result Calculation and Ranking

For each student the service computes subject grades, the semester GPA, the percentage, the status and the division; then `rankStudents` orders the cohort with merge sort and assigns competition ranks. A ten-character verification code is generated for every result row.

5.1.6 Email Service

`mailer.js` writes each message to `email_logs`, and a worker sends one queued message at a time. The mail center page shows counters (sent, queued, failed, skipped), the connection state, a test form, a filterable log and a preview of every message. When no Gmail account is configured the messages are still logged, with status skipped, so the system works in a demonstration without credentials.

5.1.7 Daily Class Automation

`routes/classroom.js` implements the timetable, class diary and assignments. When a slot is added the API checks for overlapping time on the same weekday for the same teacher or for the same program and term, and answers HTTP 409 with the name of the clashing subject. A diary entry is stored with an upsert, so saving again for the same subject and date edits the entry. The assignment tracker saves a status and optional marks for each student and checks the marks against the maximum.

5.1.8 Frontend Implementation

The client is a single page application with route level code splitting. A `Shell` component provides the sidebar, top bar, command palette, theme toggle and notification menu for every signed-in role. Shared components (`Select`, `Seg`, `Modal`, `Drawer`, `Chip`, `PageHeader`, `Field`, toast and confirm dialogs) are in one file so that every page looks and behaves the same. The marksheet component prints cleanly and carries the QR code produced by `qrcode.react`.

5.2 Testing

Testing was done at four levels so that each kind of defect is found where it is cheapest to find. All suites can be run locally by following the README.

Table 5.2: Test suites

Level	Tool	Scope	Result
Unit	node:test	Grading, ranking, heap, regression, k-means, risk score, email queue with a local SMTP server, reserved domains	18 of 18 passed
API flows	fetch (tests/flows.mjs)	Authentication, CRUD of every entity, marks, CSV import, back papers, attendance, publish, promote, public pages, a 59-endpoint by 4-role permission matrix, rate limit	58 of 58 passed
Smoke	fetch (tests/smoke.mjs)	Dashboard counts, student overview, verification, CSV import, attendance, analytics, insights	43 passed, 0 failed
Classroom	fetch (tests/classroom.mjs)	Timetable clash rules, today view, diary, assignments, permissions	32 of 32 passed
Browser	puppeteer-core with Edge	Public pages, sign in, registration, every admin, teacher and student page, forms, dialogs, shortcuts, phone and tablet widths, dark theme	56 of 58 steps passed on the best clean run

The browser suite is sensitive to the speed of the computer. On a computer with little free memory, different steps timed out in different runs, while the same steps passed when the machine was idle. For this reason the browser result is reported as a range and not as a guarantee. The browser suite did find real defects, which were fixed: the command palette stayed open after navigation, the marks page was missing an import, and the back paper screen crashed because of a missing import. These were found because the tests drive the real pages.

5.2.1 Test Cases

Table 5.3: Selected functional test cases

ID	Test	Input / action	Expected result	Result
TC1	Sign in with wrong password	Valid email, wrong password	HTTP 401 with a generic message	Pass
TC2	Teacher registration	Register as teacher	Account pending; cannot sign in until approved	Pass
TC3	Duplicate symbol number	Register with an existing symbol number	Rejected with a clear message	Pass
TC4	Marks above full marks	Internal 45 where full is 40	Rejected; nothing saved	Pass
TC5	CSV import with unknown symbol numbers	File with one valid and one unknown row	Valid row saved; skipped rows reported	Pass
TC6	GPA calculation	Three subjects of 3 credits with 67.5%, 82.5%, 80%	SGPA 3.37 (2.7, 3.7, 3.7)	Pass
TC7	Subject fails on external part	Total 45% but external below 40%	Subject failed; grade F	Pass
TC8	Back paper clears a failure	Failed subject, back paper 55%	Effective attempt is the back paper; semester GPA recalculated	Pass
TC9	Back paper fails again	Second sitting 33%	Student stays a candidate; attempt count increases	Pass
TC10	Back papers do not leak into cohort	Roster, analytics and ranking of the regular exam	Only attempt 1 rows are used	Pass
TC11	Publish results	Publish a calculated term	Students see them; emails queued	Pass
TC12	Unpublish	Unpublish a term	Students no longer see it	Pass
TC13	Verify genuine code	Open /verify/CODE	Shows genuine marksheet with student details	Pass
TC14	Verify fake code	Open /verify/AAAAAAAAAA	Shows no such marksheet	Pass
TC15	Public rate limit	More than 40 lookups in a minute	HTTP 429	Pass
TC16	Permission matrix	Each endpoint with each role	Allowed only for permitted roles (59 endpoints, 4 roles)	Pass
TC17	Reserved email domain	Send to a .test address with email configured	Logged as skipped; never delivered	Pass
TC18	Email retry	SMTP failure	Retried up to 3 times then marked failed	Pass
TC19	Timetable clash	Add a slot overlapping the same teacher	HTTP 409 with the clashing subject	Pass

ID	Test	Input / action	Expected result	Result
TC20	Teacher edits another teacher's subject	POST /timetable for a foreign subject	HTTP 403	Pass
TC21	Diary duplicate	Save the same subject and date twice	One entry, updated topic	Pass
TC22	Assignment marks above maximum	Marks 11 where maximum is 10	Rejected with HTTP 400	Pass

5.2.2 Email Verification

The email pipeline was tested without sending real messages. A local SMTP server receives a queued message and the test checks that the message is delivered and logged as sent; a second test stops the server and checks the retry and the final failed state; a third test confirms that a reserved demo domain is skipped even when email is configured. The HTML of each template was rendered in a browser and reviewed for layout (Figure A.13 in the appendix). A real Gmail account must be configured by the administrator for live delivery; the connection can be checked from the Mail center with the test message.

5.2.3 Defects Found and Fixed

Table 5.4: Notable defects found during testing

Defect	Found by	Fix
Date of birth sent as an ISO string caused a server error when a student was edited	API flow test	Normalize the date before saving
Mail retry re-queued messages even when email was not configured	API flow test	Retry does nothing when unconfigured and skips reserved domains
Confirmation dialog appeared behind a drawer	Browser test	Raised the overlay layer
The algorithms page required a login	Browser test	Made the route public
Phone layout overflowed because of fixed minimum widths	Browser test	Responsive widths and truncation of long labels
Command palette stayed on screen after choosing a page	Browser test	Removed the exit animation that never finished
Back paper screen crashed (missing import)	Visual check while preparing figures	Added the import; scanned all pages for undefined names
Seed data had a hard-coded institution name	Review	Defaults are blank; branding is optional

5.3 Result Analysis

The seeded database contains 97 students in three programs, 68 subjects, about 1,800 mark rows (including 71 back paper sittings) and about 11,700 attendance records. It was used

to check that the system behaves as expected on a realistic amount of data. With this data, the result calculation of a term of 14 students completes in a fraction of a second and every public and portal page loads within about one second on a laptop with limited memory. The ranking, back paper and permission checks give identical results on every run because the algorithms are deterministic.

The objectives of Section 1.3 were checked one by one. The configurable grading engine, back paper model, ranking and analytics, risk score, role based security with QR verification, email queue, daily class tools and neutral branding are all implemented and covered by at least one automated test. The two items that cannot be proven by the tests are conformity with the official TU regulation and live Gmail delivery; both depend on information that only the institution has.

CHAPTER 6: CONCLUSION AND FUTURE RECOMMENDATIONS

6.1 Conclusion

Anka replaces the paper ledger and spreadsheet chain of a campus with one database and a set of role based screens. Teachers enter marks once; the system calculates grades, semester and cumulative GPA, ranks the cohort, handles back paper sittings without losing history, warns about students at risk, publishes results, and sends emails. Students and the public can read results and verify printed marksheets. The daily class module extends the same platform to timetables, class diaries and assignments. The product carries no institution name, so a school, a higher secondary college or a university campus can adopt it by entering its own programs and rules.

The work met the objectives set in Chapter 1 and applied the algorithms expected from the project: sorting, searching, heap selection, statistics, regression, correlation and clustering, all written without libraries and explained in the running system. Building and testing the system also gave practical experience in database design, REST API design, security, user interface design, automated testing and documentation.

6.2 Future Recommendations

- Verify the grade table and back paper rules against the official TU regulation for each year and add rule versions so that old batches keep their old rules.
- Add multi-tenancy so that one server can host several institutions.
- Add file upload for assignments and a messaging feature between teachers and students.
- Build a mobile application and push notifications, and add SMS delivery for guardians without email.
- Add parent accounts, fee management and a document archive.
- Use real examination data (with permission) to validate the risk score weights and replace the hand-written weights with a trained model whose reasons are still shown to users.
- Make the browser test suite independent of machine speed with explicit waiting on application events, and run all suites in continuous integration.
- Export marksheets and the transcript as signed PDF documents.

REFERENCES

[1] C. Romero and S. Ventura, "Educational data mining: A review of the state of the art," *IEEE Trans. Syst., Man, Cybern. C, Appl. Rev.*, vol. 40, no. 6, pp. 601-618, Nov. 2010.

[2] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. Cambridge, MA, USA: MIT Press, 2009.

[3] J. MacQueen, "Some methods for classification and analysis of multivariate observations," in *Proc. 5th Berkeley Symp. Math. Statist. Probab.*, vol. 1, 1967, pp. 281-297.

[4] S. P. Lloyd, "Least squares quantization in PCM," *IEEE Trans. Inf. Theory*, vol. 28, no. 2, pp. 129-137, Mar. 1982.

[5] K. Pearson, "Notes on regression and inheritance in the case of two parents," *Proc. R. Soc. Lond.*, vol. 58, pp. 240-242, 1895.

[6] N. Provos and D. Mazieres, "A future-adaptable password scheme," in *Proc. USENIX Annu. Tech. Conf.*, 1999, pp. 81-91.

[7] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, Internet Engineering Task Force, May 2015.

[8] J. Klensin, "Simple Mail Transfer Protocol," RFC 5321, Internet Engineering Task Force, Oct. 2008.

[9] R. T. Fielding, "Architectural styles and the design of network-based software architectures," Ph.D. dissertation, Univ. California, Irvine, CA, USA, 2000.

[10] I. Sommerville, *Software Engineering*, 10th ed. Harlow, U.K.: Pearson, 2016.

[11] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner's Approach*, 8th ed. New York, NY, USA: McGraw-Hill, 2015.

[12] Meta Platforms, Inc., "React documentation." [Online]. Available: https://react.dev. [Accessed: 2 Nov. 2026].

[13] OpenJS Foundation, "Express: Fast, unopinionated, minimalist web framework for Node.js." [Online]. Available: https://expressjs.com. [Accessed: 2 Nov. 2026].

[14] Oracle Corporation, "MySQL 8.4 Reference Manual." [Online]. Available: https://dev.mysql.com/doc/refman/8.4/en/. [Accessed: 2 Nov. 2026].

[15] Nodemailer, "Nodemailer: Send emails from Node.js." [Online]. Available: https://nodemailer.com. [Accessed: 2 Nov. 2026].

[16] World Wide Web Consortium, "Web Content Accessibility Guidelines (WCAG) 2.1." [Online]. Available: https://www.w3.org/TR/WCAG21/. [Accessed: 2 Nov. 2026].

[17] Tribhuvan University, Faculty of Humanities and Social Sciences, "CACS452 Project III, BCA eighth semester course outline." [Online]. Available: https://www.itcollegenepal.com/wp-content/uploads/2022/02/CACS452-BCA-VIII-SEM-Project-III.pdf. [Accessed: 2 Nov. 2026].

[18] Tribhuvan University, Office of the Dean, FOHSS, "BCA curriculum." [Online]. Available: https://tufohss.edu.np. [Accessed: 2 Nov. 2026].

[19] Nepal Examination Board, "Grading system for grades 11 and 12." [Online]. Available: https://neb.gov.np. [Accessed: 2 Nov. 2026].

APPENDIX A: SCREENSHOTS

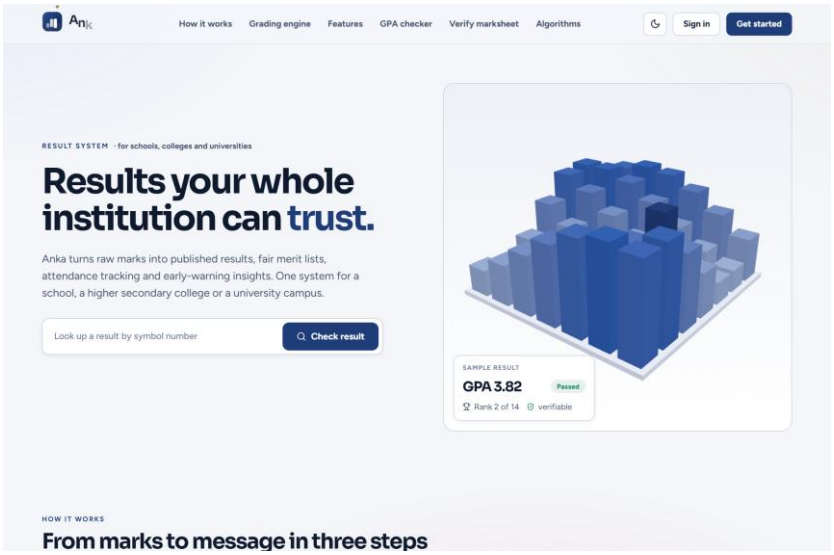


Figure A.1: Landing page

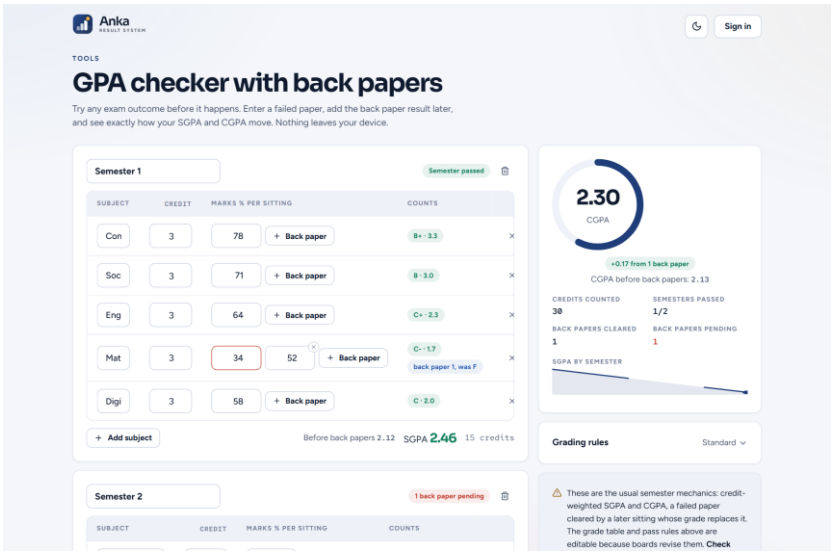


Figure A.2: Personal GPA checker with back papers

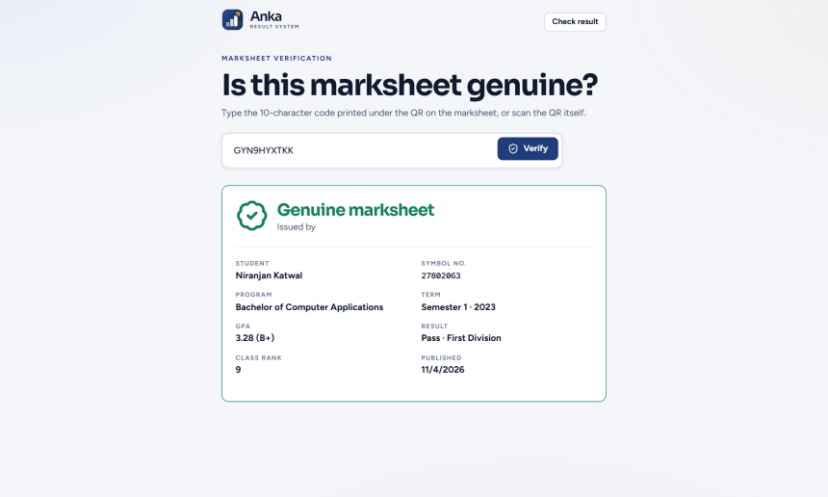


Figure A.3: Marksheet verification result

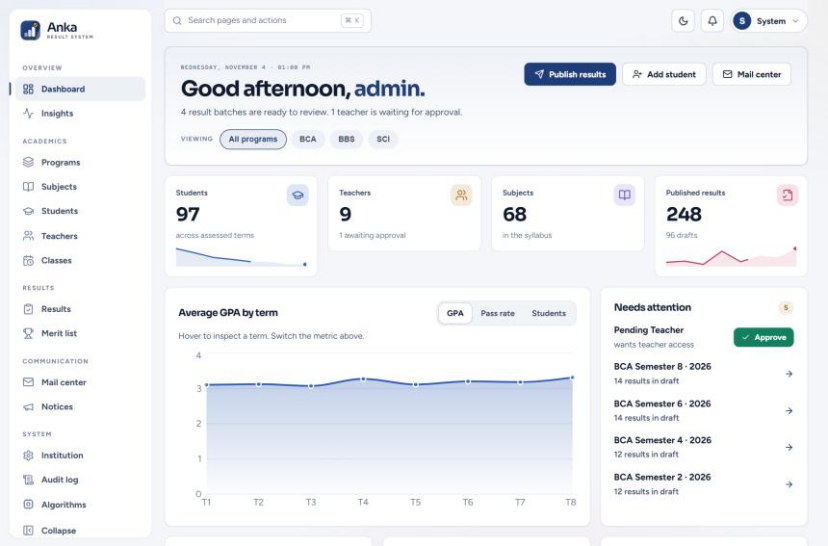


Figure A.4: Administrator dashboard

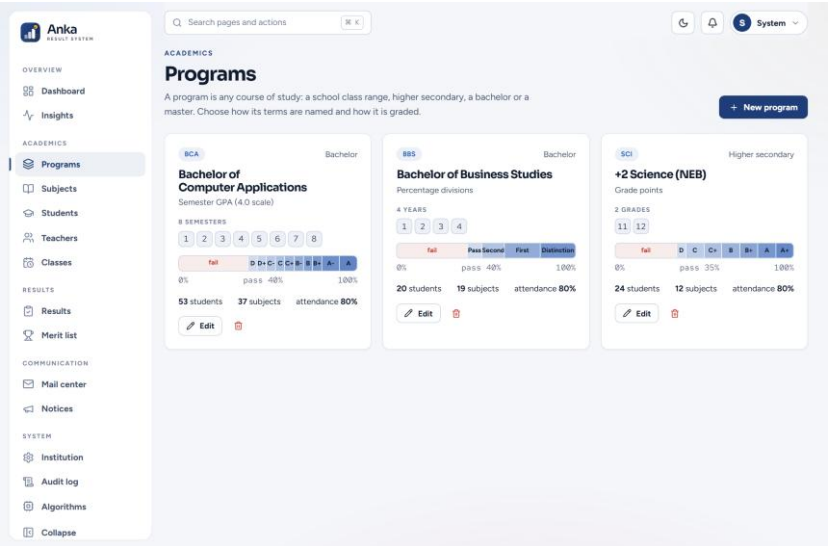


Figure A.5: Programs with grading schemes

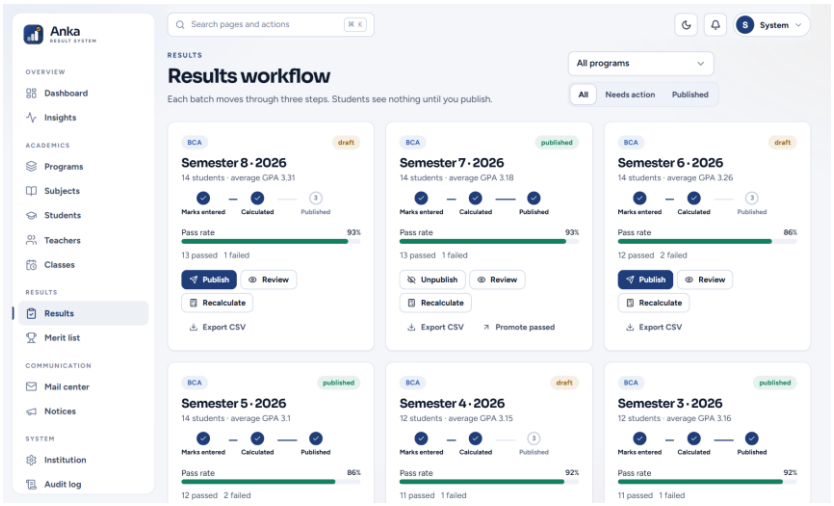


Figure A.6: Results workflow

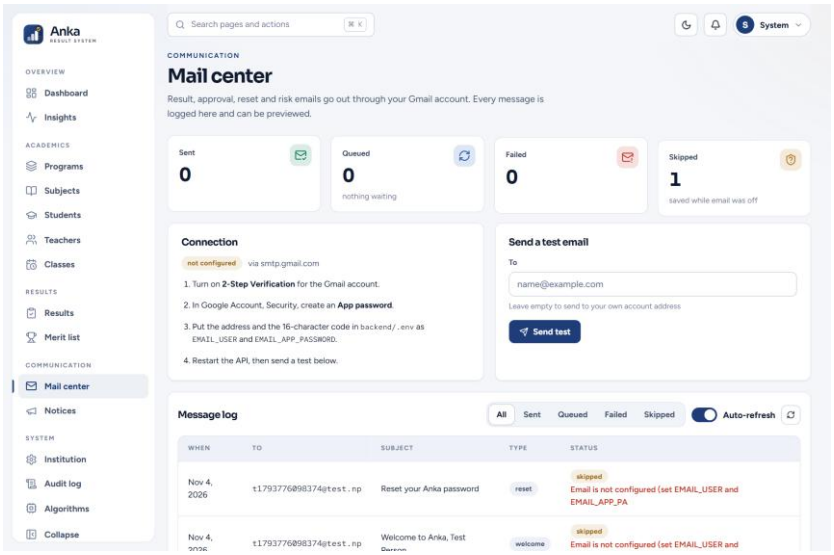


Figure A.7: Mail center

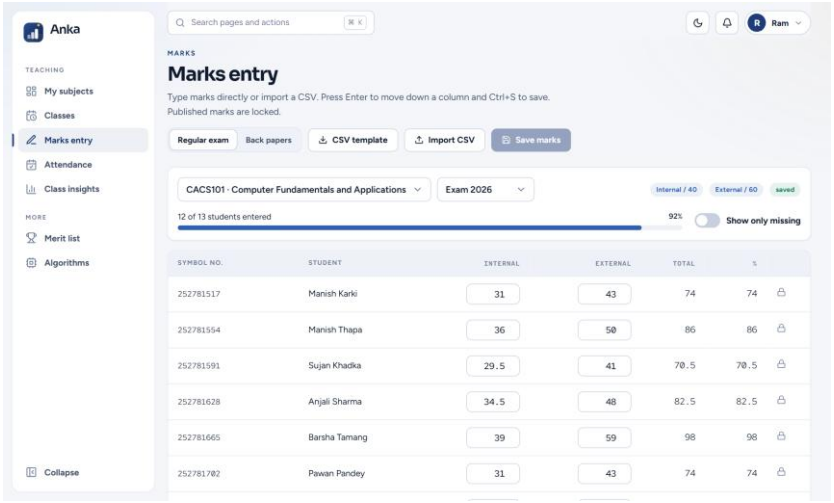


Figure A.8: Marks entry grid

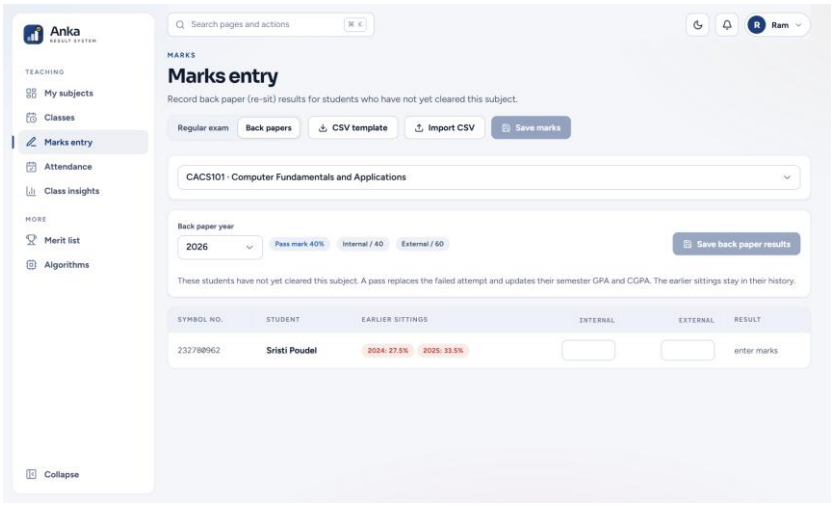


Figure A.9: Back paper entry

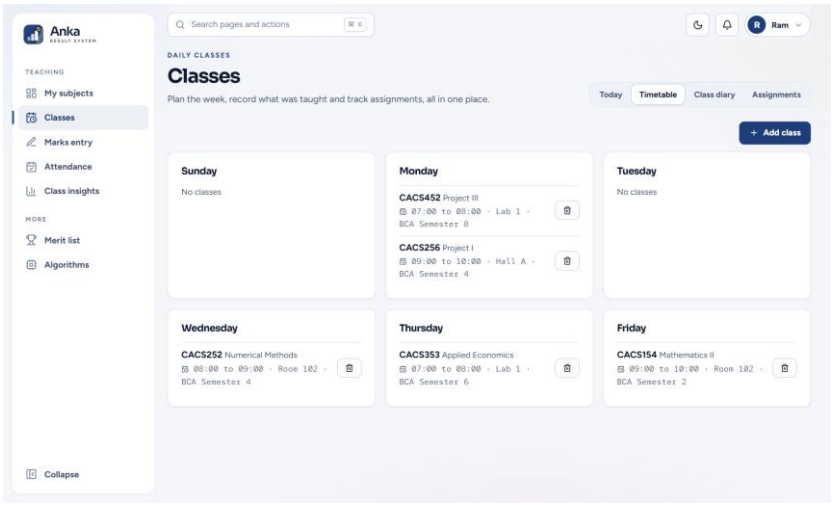


Figure A.10: Classes: weekly timetable

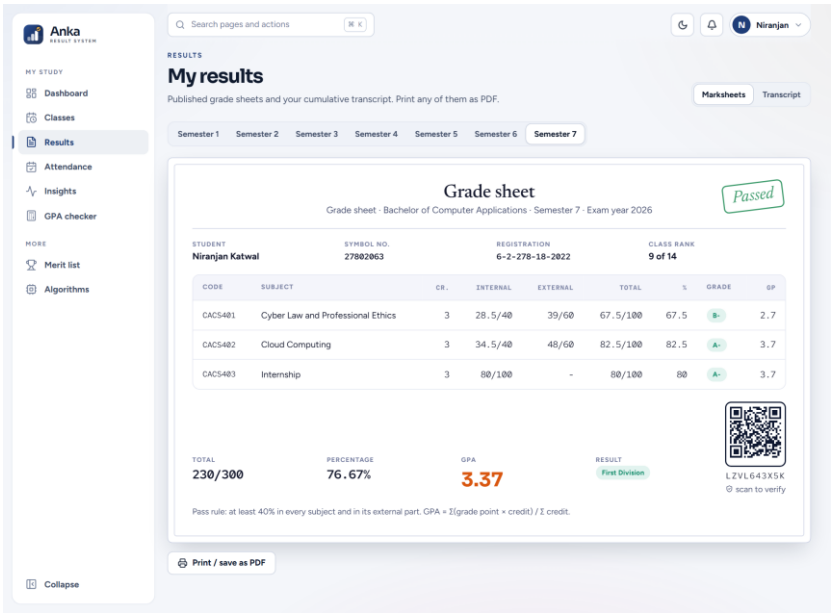


Figure A.11: Marksheet with QR code

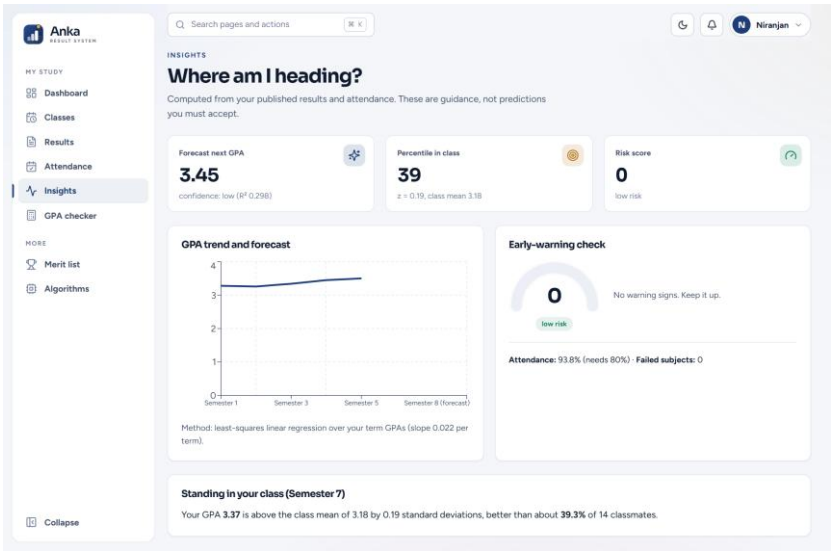


Figure A.12: Student insights and risk

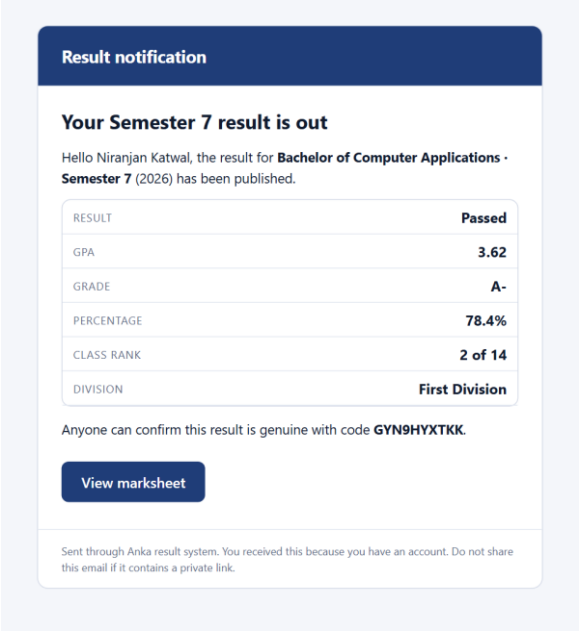


Figure A.13: Result email as rendered in a browser

APPENDIX B: RUNNING THE SYSTEM

1. Install Node.js 20 or later. Run `setup-mysql.cmd` once to prepare the portable MySQL copy, then `start-mysql.cmd` to start it (port 3308).
2. In `backend`, copy `.env.example` to `.env`, set `JWT_SECRET`, and optionally set `EMAIL_USER` and `EMAIL_APP_PASSWORD` for Gmail.
3. Run `npm install` in `backend`, `frontend` and `e2e`, then `npm run db:setup` in `backend` to create the schema and demo data.
4. Start the API with `node server.js` (port 5000) and the client with `npm run dev` in `frontend` (port 5191).
5. Sign in with the demo accounts printed by the seed script (administrator, teacher and student).

The complete source code is available in the public repository https://github.com/itsniranjannn/StuResSystem.